KMail

BADI VENKATA PRASANNA URK23AI1122 <badivenkata@karunya.edu.in>

(no subject)

1 message

BADI VENKATA PRASANNA URK23AI1122 <badivenkata@karunya.edu.in> Fri, Nov 13, 2026 at 12:55 AM
To: URK23AI1126 aalkha <aalkhak@karunya.edu.in>, VISHNUPRIYA S V URK23AI1146 <vishnupriyasv@karunya.edu.in>, BADI VENKATA PRASANNA URK23AI1122 <badivenkata@karunya.edu.in>

Exp 1: Git Essentials

```
mkdir git-essentials-lab
cd git-essentials-lab
git init
git config user.name "YourGitHubUsername"
git config user.email "you@example.com"
"# git-essentials-lab" | Out-File README.md -Encoding ascii
git add .
git commit -m "Initial commit of the project"
git branch -M main
git checkout -b feature/new-feature
Add-Content README.md "feature work"
git commit -am "Add feature work"
git checkout main
git merge feature/new-feature
git log --oneline --graph --all
git remote add origin https://github.com//git-essentials-lab.git
git push -u origin main
git push -u origin feature/new-feature
```

How to run

1. Open PowerShell and paste the first block, line by line. Put your own GitHub username and email in the two git config lines.
2. Check that git log --oneline --graph --all shows the merge.
3. In the browser: GitHub, New repository, name it git-essentials-lab, leave 'Add README' unticked, click Create.
4. Back in PowerShell, paste the second block, using your username in the URL.
5. On GitHub, open a pull request from feature/new-feature into main, then merge it.

Expected: the graph shows both branches joined, and the repository page lists README.md.

Exp 2: Flask App with Docker and Docker Hub

Create a folder flask-docker-app with these two files.

app.py

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def home():
    return "Hello Docker"

app.run(host='0.0.0.0', port=5000)
```

Dockerfile

```
FROM python:3.9-slim
COPY app.py .
```

```
RUN pip install flask
CMD ["python","app.py"]
```

```
docker build -t klassy01/flask-docker-app .
docker run -p 5000:5000 klassy01/flask-docker-app
docker login
docker push klassy01/flask-docker-app
```

How to run

1. Start Docker Desktop and wait until it says it is running.
2. In flask-docker-app, run docker build -t klassy01/flask-docker-app ..
3. Run docker run -p 5000:5000 klassy01/flask-docker-app, then open http://localhost:5000 and check the text appears. Press Ctrl+C to stop.
4. Run docker login, then docker push klassy01/flask-docker-app.

Expected: the browser shows Hello Docker, and the image appears in your Docker Hub repositories.

Exp 3: CI Pipeline with GitHub Actions and Docker Hub

Create a folder python-ci-docker-lab with these files.

app.py

```
def add(a, b):
    return a + b
```

test_app.py

```
from app import add

def test_add():
    assert add(2, 3) == 5
```

.github/workflows/ci.yml

```
name: ci

on:
  push:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install pytest && pytest
```

```
git init
git add .
git commit -m "init lab"
git branch -M main
git remote add origin https://github.com//python-ci-docker-lab.git
git push -u origin main
```

How to run

1. Create the folder and the three files above: app.py, test_app.py, and the workflow at .github/workflows/ci.yml.
2. On GitHub, create an empty repository named python-ci-docker-lab.
3. Run the git block above. The push starts the workflow.
4. Open the repository's Actions tab and check that the run is green.

Expected: the Actions tab shows a green run with the pytest step passed.

Exp 4: Jenkins Pipeline for a Static Website

Create a folder jenkins-static-site with one file. The pipeline script is pasted into Jenkins, not saved as a file.

docker-compose.yml

```
services:
  jenkins:
    image: jenkins/jenkins:lts
    ports: ["8080:8080"]
```

Pipeline script

```
pipeline {
    agent any

    stages {
        stage('Generate') {
            steps {
                sh 'mkdir -p site && echo "# Karunya" > site/index.html'
            }
        }
    }

    post {
        always {
            archiveArtifacts artifacts: 'site/**'
        }
    }
}
```

How to run

1. In jenkins-static-site, run
2. Run docker compose up -d.
3. Run docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword and copy the password.
4. Open http://localhost:8080, paste the password, choose Install suggested plugins, and create the admin user.
5. Click New Item, choose Pipeline, and name it karunya-site-pipeline.
6.
7. Click Build Now and wait for the green tick.
8. Clean up with docker compose down.

Expected: the build is green and site/index.html is listed under Build Artifacts.

Exp 5: Deployment of a Reactive Web Page Using Jenkins and Docker

docker-compose.yml

```
version: '3.8'

services:
  jenkins:
    image: jenkins/jenkins:lts
    container_name: jenkins-exp4
    ports:
      - "8090:8080"
      - "50001:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock

volumes:
  jenkins_home:
```

Jenkinsfile

```
pipeline {
    agent any

    stages {
        stage('Generate Site') {
            steps {
                script {
                    sh 'rm -rf site && mkdir -p site/assets'

                    writeFile file: 'site/index.html', text: '''
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,initial-scale=1">
    <title>Karunya University</title>
    <link rel="stylesheet" href="assets/style.css">
</head>
<body>
    <header>
        <h1>Karunya University</h1>
        <p>Values - Knowledge - Service</p>
    </header>

    <main>
        <h2 id="about-title"></h2>
        <p id="about-text"></p>
        <button onclick="updateContent()">Update Content</button>
    </main>

    <footer>Generated by Jenkins</footer>
    <script src="assets/app.js"></script>
</body>
</html>
'''

                    writeFile file: 'site/assets/style.css', text: '''
body { font-family: Arial; margin: 20px; }
header { background: #0b3d91; color: white; padding: 20px; }
main { padding: 20px; }
button { padding: 8px; }
footer { margin-top: 20px; }
'''

                    writeFile file: 'site/assets/app.js', text: '''
let state = {
    title: "About Karunya University",
    description: "Karunya Institute of Technology and Sciences is a leading centre for higher
education in India."
};

function render() {
    document.getElementById("about-title").innerText = state.title;
    document.getElementById("about-text").innerText = state.description;
}

function updateContent() {
    state.description = "Content updated dynamically without page reload!";
    render();
}

document.addEventListener("DOMContentLoaded", render);
'''
                }
            }
        }

        stage('Publish Site') {
            steps {
                publishHTML([
                    reportDir: 'site',
                    reportFiles: 'index.html',
                    reportName: 'Karunya University - Reactive Site',
```

```
                    keepAll: true,
                    alwaysLinkToLastBuild: true,
                    allowMissing: false
                ])
            }
        }
    }

    post {
        always {
            archiveArtifacts artifacts: 'site/**', fingerprint: true
        }
    }
}
```

How to Run

1. Create a folder:

```
mkdir jenkins-static-site
cd jenkins-static-site
```

2. Create `docker-compose.yml` and paste the Docker Compose code.

3. Start Jenkins:

```
docker compose up -d
```

4. Get the Jenkins password:

```
docker exec jenkins-exp4 cat /var/jenkins_home/secrets/initialAdminPassword
```

5. Open:

http://localhost:8090

6. Enter the password.

7. Select **Install suggested plugins**.

8. Create the Jenkins admin user.

9. Click **New Item**.

10. Enter:

```
karunya-site-pipeline
```

11. Select **Pipeline** → **OK**.

12. Scroll to **Pipeline** → set **Definition** to:

```
Pipeline script
```

13. Paste the complete `Jenkinsfile` above into the **Script** box.

14. Click **Save**.

15. Click **Build Now**.

16. After the build, open the build → **Artifacts** / **Build Artifacts**.

17. To clean up:

```
docker compose down
```